

web3d-lab • 3DGS 在线重建查看器 + 个人主页

郑州大学 双足实验室（智能机器人创新训练基地）招新第一轮考核作品 · 方向：3D（三维视觉 / 三维重建）

作者：肖宇航（软件工程 2026 级） 线上：www.shawnlab.cn 源码：github.com/Shawn100861224/web3d-lab

联系：18711505157@163.com

这是什么

一个前后端一体的作品站。访客打开网页，能在浏览器里实时旋转我用手机拍的真实物体训练导出的三维高斯泼溅（3DGS）场景，并在旁边看到该场景的真实训练指标（PSNR / SSIM / 高斯点数 / 训练耗时 / 显存占用）——这些数字由训练脚本回写进数据库，不是写死的。考核里多数人交静态个人主页；能把「自己拍、自己训出来的重建场景」放进网页实时渲染的，几乎没有，这是这个作品的全部立足点。

六个页面

首页	精选场景 + 访问统计（14 天曲线）	场景库	卡片列表，缩略图 / 点数 / 指标，数据来自接口
查看器	Three.js + Spark 加载 .spz / .splat，轨道漫游 + 自动巡航 + 实时指标面板	方法对比	3DGS 家族 9 类方法 × 8 个维度（基元/表面/规模/速度/压缩/开源/“为什么和我有关”）
关于我	方向理解、真实做过的事、踩过的坑、116 个仓库的项目雷达（真实抓取数据）、用 AI 的过程、下一步	留言板	可写（有后端时，含限流与注入防护）；静态部署下展示离线快照并标注边界

技术栈

前端	Vite 8 + React 19 + TypeScript	3D 渲染	Three.js + @sparkjsdev/spark
后端	FastAPI + SQLAlchemy + SQLite (Python 3.11)，4 组接口	训练	COLMAP (SfM) + gsplat；本机 RTX 5060 Laptop 8GB，重活租云端 GPU (AutoDL vGPU-32GB)
测试	pytest（后端 36 项）+ 截图像素级渲染核验脚本	部署	自有域名 www.shawnlab.cn （腾讯云 EdgeOne · 免备案）+ GitHub Pages 镜像，CLI 自动部署

明确不用 Electron：交付形态是「一个链接」——安装包手机打不开、对 3D 零增益。

我具体做了什么（AI 做了什么、我否掉了什么）

- 先定结构再让 AI 写：先想清页面要回答评审的四个问题，把每节内容写成中文要点，再让 AI 实现版式——反过来会得到漂亮但没有信息的页面。
- 不让 AI 编数据：星标是脚本抓的；3DGS 的点数、PSNR、帧率是跑出来的。
- AI 的方案被我退回两次：项目雷达先用爬虫抓 GitHub → 我改成官方 API；接口限额没处理 → 改成已授权令牌 + 结果落文件。
- 发现筛选逻辑不严，自己补规则：抓回 138 个仓库里混着人脸识别、扫地机器人等不相关项目 → 我加「相关性关键词 + 人工否决清单」，重抓到 116 个。
- 不接受“跑起来了就算完成”：要求每步留下证据——注册率不足就报错、空画布不许算通过（用像素统计判定）。
- 主动放弃并记录：全分辨率 30000 步实测每步 3.6 秒（约 30 小时）→ 停掉，把数字记进文档，而不是硬跑。

怎么验证的（都是真实运行的输出，不是“应该没问题”）

39/43 实拍照片 COLMAP 注册率 (90.7%)	30,000 步 云端 GPU 训练步数	66,904 训练产出的高斯点数	14.44 dB 留出视角 PSNR (8 视角)
659 s 云端全分辨率训练耗时	36 项 后端自动化测试		

- 训练链路端到端跑通：手机照片 → COLMAP 稀疏重建 → 云端 GPU 上 gsplat 训练 30000 步（渲染分辨率 2016×1512）→ 导出上网版资产 → 网页实时渲染 → 指标回写数据库。
- 三组对照实验，不是无限调参：同一批数据在云端跑三组（半分辨率全量稠密化 64.8 万高斯 / 全分辨率保守稠密化 1.5 万高斯 / 全分辨率中等稠密化 6.7 万高斯），留出视角 PSNR 分别为 14.46 / 14.52 / 14.44 —— 显存从 1.5 GB 换到 31 GB，画质几乎没动，据此把瓶颈定位到拍摄而非算力，避免了继续烧钱。
- 查出一个真实的评估缺陷并修掉：原先「留出 8 个视角」取的是最后 8 个，实测它们是一整段连续弧（约占环绕 70°）→ 这一段完全没有参与训练，而网页查看器是让访客自由环绕的，第一眼就撞上盲区。改成按相机绕主体的方位角均匀留出后，留出的 8 个视角散开到整圈。
- 数据一致性可对账：数据库里的高斯点数 × 32 字节 = 资产实际字节数，逐字节吻合；上线后用 md5 比对线上与本地资产，两处入口一致。
- 像素级判空：用「亮像素占比 + 标准差 + 重心」区分“真渲染出来”和“加载成功但全黑”。
- 后端是真的：空库返回 200 而非 404、重复 slug 409、未知场景 404、限流 429 + Retry-After、UTF-8 往返、时区分桶、CORS 白名单——都有对应测试；XSS 载荷实测只以文本呈现、DOM 无注入节点。

已知边界（不想含糊过去）

- 本机显存上限（实测，它决定了能做什么）：WSL 下 GPU 单进程可分配显存只有约 1.5 GB（标称 8 GB；驱动报的“空闲”数不可信），且与 Windows 可用内存强相关（实测 700 MB ↔ 2.7 GB）。结论：单物体 / 桌面级可行（3–5 万高斯即可），房间级不行（参考实现需 100–300 万高斯）。已用公开数据集（Mip-NeRF 360，240 张真实照片、自带 COLMAP 位姿）把同一条管线跑通验证：流程全绿、跳过 SfM 只要 15 分钟，但只塞得下 6.6 万高斯，画质糊（PSNR 11.6），因此主动没有上线——证据与数据都在仓库里。
- 线上是静态演示版：留言板与访问统计需要后端服务，而公网后端需要自有域名（域名已就绪）。后端代码完整可跑，本地一条命令启动。
- 自训场景的画质瓶颈在拍摄，不在算力（这是我这轮最想讲清楚的结论）：同一批 43 张手机照片（COLMAP 只注册 39 张，其中 6 张被清晰度检查判为糊片，实际参与训练 31 个视角），在云端 31 GB 显存上跑满 30000 步、全分辨率渲染，留出视角 PSNR 仍只有 14.4 —— 与 8 GB 本机同量级。网页里能认出是鞋，但边缘偏软、背景杂点明显；同一个查看器加载官方示例（.spz 资产）是清晰的，说明问题不在查看器或部署。真正的解法是重拍（视角加密到 50–80 张、避免糊片、贴近平移、背景干净），而不是加高斯、加步数或换更大的卡——这三条我都实测否掉了。
- 展示中的实拍场景就是我自己拍的运动鞋：43 张手机环拍 → 降采样与亮度归一 → COLMAP → 云端 30000 步训练 → .splat 上网。另有一个 15 个视角的「保温杯」场景作为受限对照（PSNR 10.4），用来佐证“输入决定上限”。

下一步

- 重拍一轮（50–80 张、锁定曝光、避免糊片、背景干净）→ 用同一套管线再训一次，把线上实拍场景的画质提上去。
- 把后端放到自有域名下（EdgeOne Functions 原生支持 Python/FastAPI，同域免 CORS），让访问统计与留言板真正上线。
- 按实验室安排打 C++ / Linux 基础：WSL2 环境已就绪（首个 C++ 程序与 CMake 多文件项目已跑通）。

仓库内含全部过程记录：`README.md`（技术栈与验收标准）、`PROGRESS.md`（模块进度与证据）、`HANDOFF.md`（每轮的结论、花费与坑表）、`pipeline/README.md`（训练管线）。本轮「云端 30000 步」的三组对照数据、留出视角缺陷的定位过程、以及部署踩坑都写在里面，可随时按文档复现。